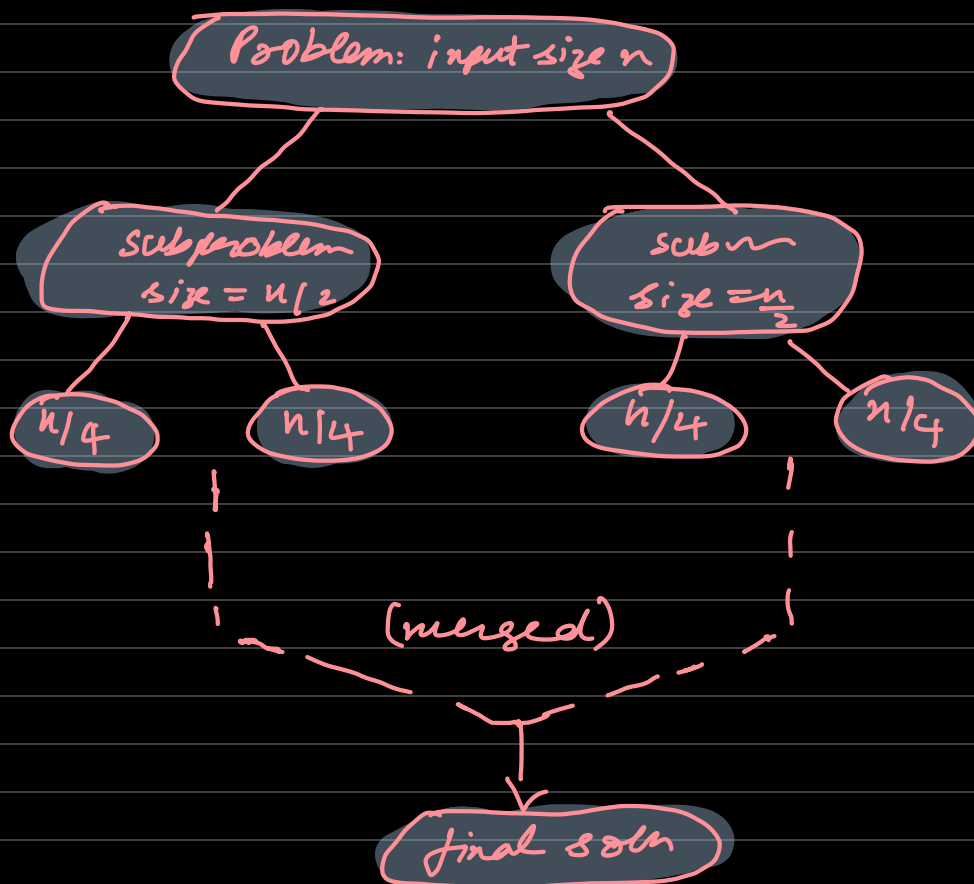


Unit II:

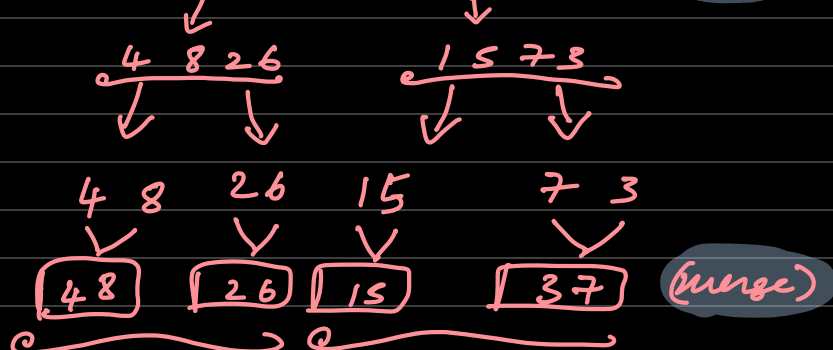
Merge sort: divide & conquer

⇒ prob is divided into 2 parts and each part is solved recursively and merged together.

⇒



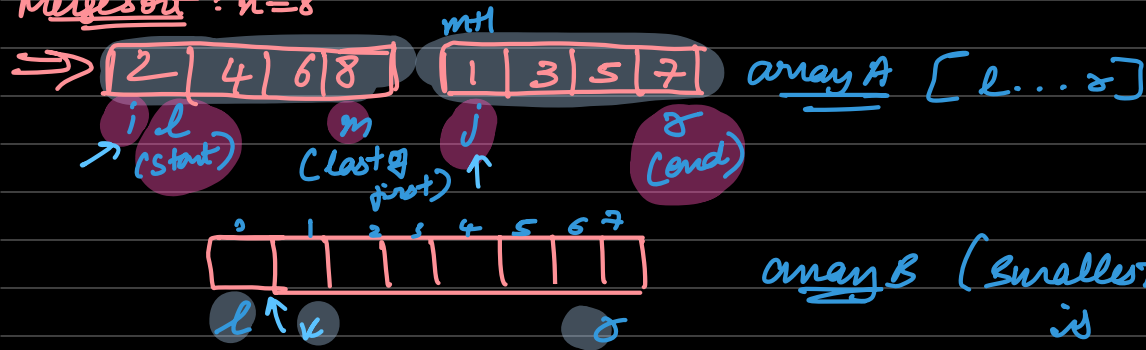
ex: 4 8 2 6 1 5 7 3 (sort)



2 4 6 8 1 3 5 7

1 2 3 4 5 6 7 8 (store in diff array)

Merge sort: $n=8$



$\Rightarrow$ merge array A and B.

$i \leftarrow l$
 $j \leftarrow m+1$ } array A
 $k \leftarrow l$ (new array)

$\Rightarrow$ compare $A[i]$ with $A[j]$

if $A[i]$ is less, copy to $B[k]$ and increment i & k to next
else $A[j]$ is less, copy to $B[k]$ j & k to next

$\Rightarrow$ the last ele remaining in j but not in i then copy to B is remaining to B .

$\Rightarrow$ copy all ele of B to A

(Bubble sort & selection sort is inplace
merge sort is not inplace as another array is used).

Algorithm merge sort (A, l, m, r)

// input: An array A with l & r as 1st & last ele.

// output: sorted array A into 2 sorted arrays

if $l < r$

$m \leftarrow l + r/2$

merge sort (A, l, m)

merge sort ($A, m+1, r$)

merge (A, l, m, r)

Algorithm: Merge (A, l, m, r)

// merges two sorted arrays

// input: 2 sorted arrays $A[l \dots m]$ and $A[m+1 \dots r]$

// output: sorted array.

$i \leftarrow l$

$j \leftarrow m+1$

$k \leftarrow l$

while $i \leq m$ and $j \leq r$

if $A[i] < A[j]$

$B[k] \leftarrow A[i]$ (update i & k)

else

$A[j] < A[i]$ (update $j \leftarrow i$)

$B[k] \leftarrow A[j]$

while $i \leq m$

$B[k] \leftarrow A[i]$ (update $i \leftarrow k$)

while $j \leq r$

$B[k] \leftarrow A[j]$ (update $j \leftarrow k$)

Time complexity:

① Comparison is B.O.P.

② $C(1) = 0$: Base case (if $n=1$, no comparison as it is alone)

③ $C(n) = 2 C(n/2) + C_{\text{worst case}}$

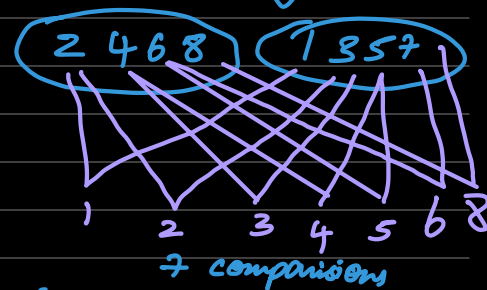
$$= 2 C(n/2) + (n-1)$$

$\Rightarrow$ By M.T:

$$a=2 \quad b=2 \quad d=1$$

$$b^d = 2$$

$$\therefore a = b^d \therefore T(n) = \theta(n \log n)$$

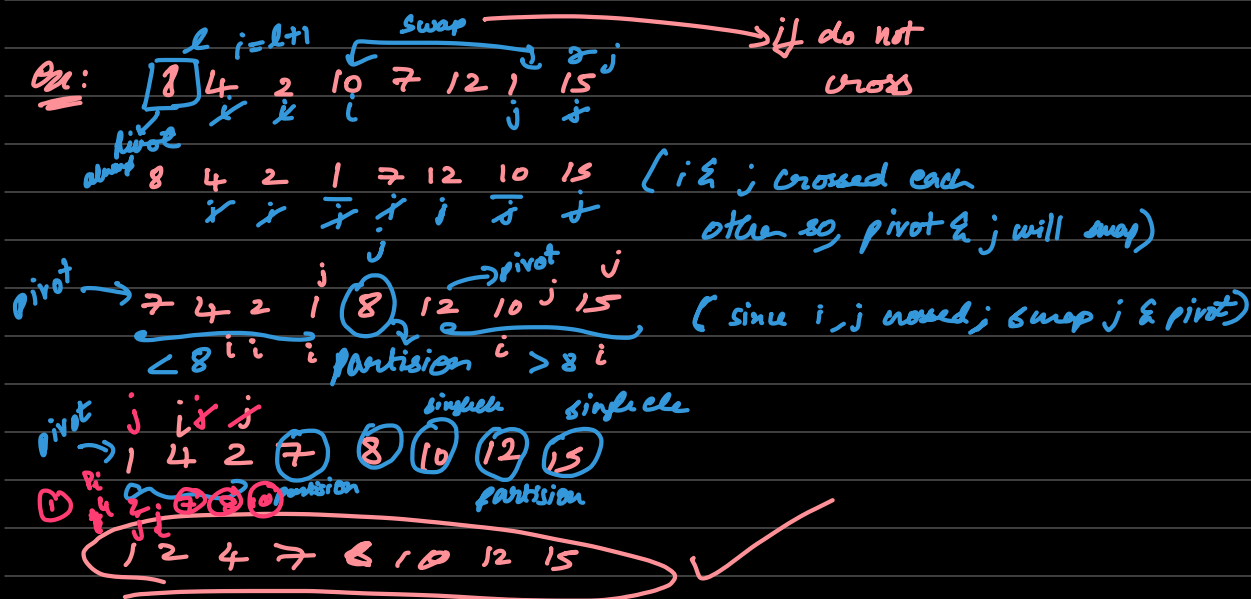


($n=8$ but no. of max comparisons made is $n-1$)

Quicksort:

$\Rightarrow$ Based on partition (we choose either first or last ele as pivotele)

$\Rightarrow$ In the end of partition, the pivot ele is placed in its actual position. All the ele equal to it or left to it are smaller than it.



Algorithm quicksort (A, l, r)

// recursion to sort ele using Q.S

// input: $\sim$

// output: $\sim$

if $l < r$

$s \leftarrow \text{partition}(A, l, r)$

Quicksort($A, l, s-1$)

Quicksort($A, s+1, r$)

$\Rightarrow s \Rightarrow \text{position of partition}$

$\Rightarrow [l \quad s \quad r]$

Algorithm partition (A, l, r)

// partitions a sub array.

// in: A -array.

// out: places pivot ele in actual position

pivot $\leftarrow A[l]$

$i \leftarrow l+1$

$j \leftarrow r$

while True

while pivot $\geq A[i]$

increment i

while pivot $< A[j]$

decrement j

if $i < j$ (if not crossed)

Swap ($A[i]$ and $A[j]$)

else

Swap ($A[l]$ & $A[j]$)

return j

Time Complexity:

any case $\Rightarrow 1.38 \cdot \log n$

$\Rightarrow$ Comparison.

$\Rightarrow$ Best case: (when pivot div equally)

worst case:

$\Rightarrow$ Best case:

$$C(n) = 2C(n/2) + n$$

n.o. of comparisons
done to put pivot
here.

$$a = 2 \quad b = 2 \quad d = 1$$

$$b^d = 2$$

$$\therefore a = b \quad \therefore T(n) = \Theta(n^d \log n) = \underline{\underline{\Theta(n \log n)}}$$

$\Rightarrow$ worst case: $C(n) = (n+1) + n + (n-1) + \dots + 3$ (here size = 3, we have 3

$$= (n+1) + n + (n-1) \dots 3+2+1-1-2 \text{ comparisons always)$$

$$= \frac{(n+1)(n+2)}{2} - 3$$

$$T(n) = \underline{\underline{O(n^2)}}$$

Quadratic

$$\frac{n(n+1)}{2}$$

$$\frac{(n+1)(n+1)}{2}$$

$$\frac{(n+1)(n+2)}{2}$$

Multiplication of 2 n digit no. : divide & conquer

⇒ let a & b be 2 integers. $C = a \times b$

⇒ $\left. \begin{array}{l} a = a_1 a_0 \\ b = b_1 b_0 \end{array} \right\} \begin{array}{l} \text{size of } a \Rightarrow 2 \text{ digits} = 2 \\ \text{size of } b \Rightarrow 2 \text{ digits} = 2 \end{array} \right\} \text{size of } a \times b = 4$

$n = 2$

formulas :

$$C_0 = a_0 \times b_0 \rightarrow 1$$

$$C_1 = (a_0 + a_1)(b_0 + b_1) - (C_2 + C_0) \rightarrow 1$$

$$C_2 = a_1 \times b_1 \rightarrow 1$$

$$C = C_2 \cdot 10^n + C_1 \cdot 10^{n/2} + C_0 \rightarrow 0$$

ex (2)

$$a = 1212$$

$$a_1 = 12$$

$$a_0 = 12$$

$$b = 0322$$

$$b_1 = 03$$

$$b_0 = 22$$

$$C_0 = 12 \times 22 = 264$$

$$C_2 = 12 \times 3 = 36$$

$$C_1 = 24 \times 25 = 300$$

$$C = 36 \cdot 10^4 + 300 \cdot 10^2 + 264$$

$$= 36000 + 30000 + 264$$

$$= \underline{\underline{390264}}$$

ex: $a = 26 \quad b = 45$
 $a_1 = 2 \quad b_1 = 4$
 $a_0 = 6 \quad b_0 = 5$

$$C_0 = 6 \times 5 = 30$$

$$C_2 = 2 \times 4 = 8$$

$$C_1 = (2+6) \times (4+5) - (8+30)$$

$$= 8 \times 9 - 38$$

$$= 72 - 38 = \underline{\underline{34}}$$

$$C = 8 \cdot 10^2 + 34 \cdot 10^{2/2} + 30$$

Time Complexity:

⇒ By M.T:

$$C(n) = 3M(n/2) + C_n$$

$$a = 3 \quad b = 2 \quad d = 1$$

$$b^d = 2$$

$$\therefore a > 3 \quad \therefore T(n) = \Theta(n \log_b a) = \Theta(n \log_2 3) = \underline{\underline{\Theta(n^{1.59})}}$$

3ML

Strassen's matrix multiplication:

⇒ Based on divide & conquer.

Ex:

$$A = \begin{bmatrix} A_1 & A_2 \\ A_3 & A_4 \end{bmatrix} \quad B = \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix} \quad C = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix}$$

$$\underline{C = A \times B}$$

⇒ formulas:

$$\begin{aligned} D &= A_1 (B_2 - B_4) & 1 \\ E &= A_4 (B_3 - B_1) & 2 \\ F &= B_1 (A_3 + A_4) & 3 \\ G &= B_4 (A_1 + A_2) & 4 \\ H &= (A_3 - A_1) (B_1 + B_3) & 5 \\ I &= (A_2 - A_4) (B_3 + B_4) & 6 \\ J &= (A_1 + A_2) (B_1 + B_4) & 7 \end{aligned}$$

⇒ final matrix:

$$C_1 = E + I + J - G$$

$$C_2 = D + G$$

$$C_3 = E + F$$

$$C_4 = D + H + J - F$$

$M(n) \Rightarrow$ no. of mult done.

$$\Rightarrow 7M(n/2) + C_n$$

Base case $\Rightarrow M(1) = 1$ (if one elem, no. of mult is 1)

By M.T:

$$a = 7 \quad b = 2 \quad d = 1$$

$$b^d = 2$$

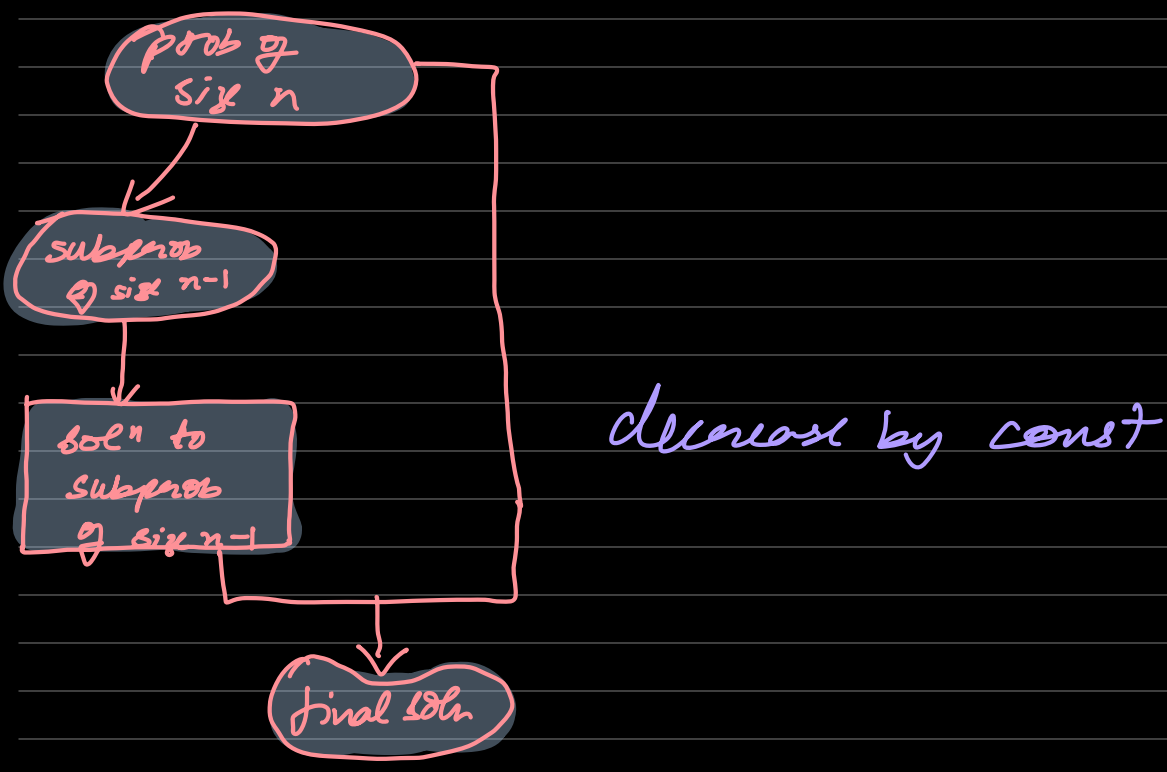
$$\therefore a > b^d \quad \therefore T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 7}) = \underline{\underline{\Theta(n^{2.81})}}$$

Insertion sort: Insertion sort, DFS, BFS

⇒ decrease & conquer. ⇒ $(n-1)$.

⇒ three variants of decrease & conquer:

- ① decrease by a constant (decrease by one) (DFS, BFS)
- ② decrease by const factor (take coin problem, Josephus prob)
- ③ variable size decrease (to find median (partition))



Ex: 24 12 8 15 20

12 24 8 15 20 1st iteration

12 8 24 15 20

8 12 24 15 20 2nd

8 12 15 24 20 3rd

8 12 15 20 24 48

Algorithm insertion sort ($A[0 \dots (n-1)]$)

// array

// array

// sorted arr.

for $i \leftarrow 1$ to $n-1$ do

$v \leftarrow A[i]$

$j \leftarrow i-1$

while $j \geq 0$ and $A[j] > v$ do

$A[j+1] \leftarrow A[j]$

$j \leftarrow j-1$

$A[j+1] \leftarrow v$

	$j+1$				
0	1	2	3	4	5
6	5	4	3	2	1
j	i				(n-1)
	v				

Time Complexity:

Best case $\Rightarrow$ arr 1 2 3 4 5 6 $\Rightarrow$ if all ele are already sorted, we do one comparison only.

$$C(n) = \sum_{i=1}^{n-1} 1 = n-1 + 1$$

$$= n-1 = \underline{\underline{O(n)}}$$

Worst case $\Rightarrow$ descending order $\Rightarrow (n-1)$ comparisons.

ex: 6 5 4 3 2 1

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} = \sum_{i=1}^{n-1} i + 1$$

$$= \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \underline{\underline{O(n^2)}}$$

DFS:

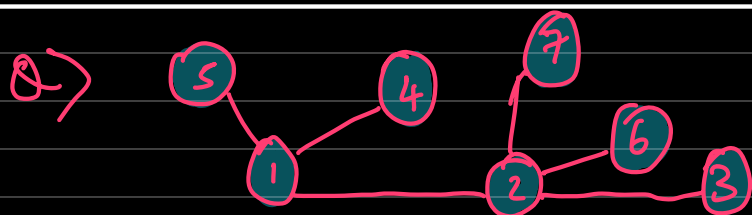
Depth first search (6)

- ⇒ uses **Stack data stmc.**
- ⇒ **edge-based technique.**
performs two stages, first visited vertices are pushed into the stack. Second, if no vertices are present, the visited vertices are popped
- ⇒ **fast process / traversing**
- ⇒ works on the concept **LIFO**
- ⇒ Builds the tree **sub-tree by subtree.**
- ⇒ **less memory req.**

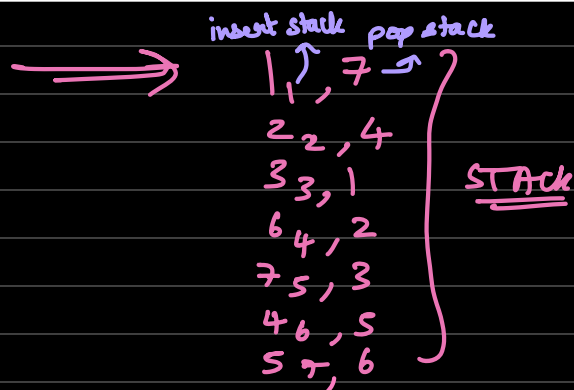
Breadth first search

- ⇒ uses **queue data stmc.**
- ⇒ **vertex-based technique.**
one vertex is selected at a time & when it is visited & marked then its adjacent are visited & stored in the queue.
- ⇒ **slower than dfs.**
- ⇒ ~ ~ ~ **FIFO**
- ⇒ Builds the tree **level by level.**
- ⇒ **more mem required**

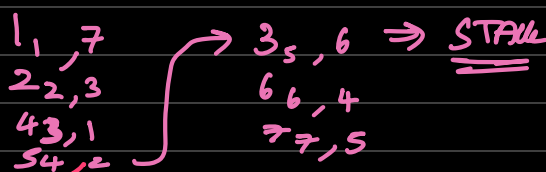
Both are used for traversing & searching graphs and trees.



⇒ DFS: 1, 2, 3, 6, 7, 4, 5



⇒ DFS: 1, 2, 4, 5, 3, 6, 7

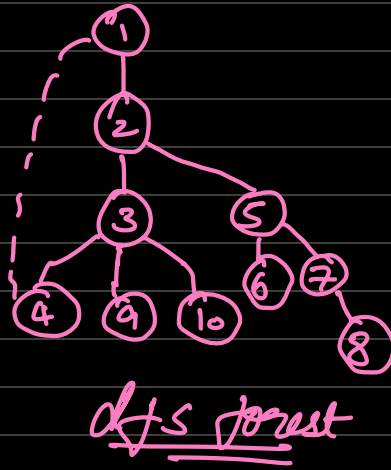


Q)

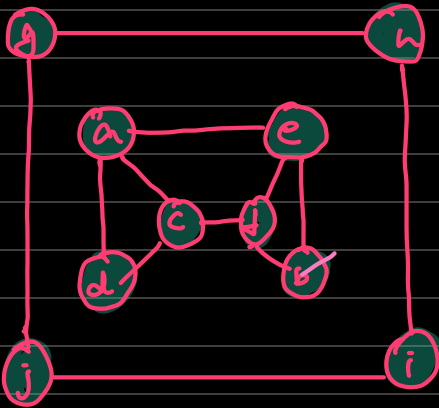


DFS:

1 1, 10
2 2, 9
3 3, 4
4 4, 1
9 5, 2
10 6, 3
5 7, 8
6 8, 5
7 9, 7
8 10, 6

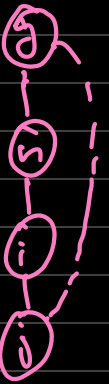
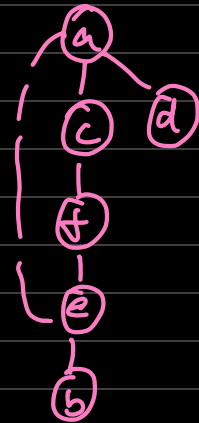


Q)

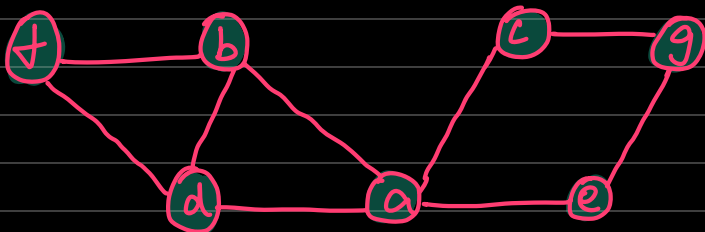


DFS:

a 1, 6
c 2, 4
f 3, 3
e 4, 2
b 5, 1
d 6, 5
g 7, 10
h 8, 7
i 9, 8
j 10, 7

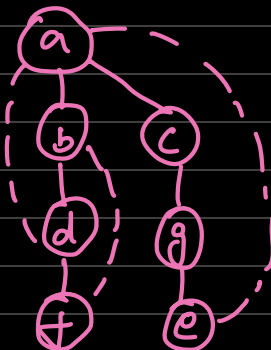


Q)



Stack
a 1, 7
b 2, 3
d 3, 2
f 4, 1
c 5, 6
g 6, 5
e 7, 4

forest



Tree edges $\Rightarrow$ edge b/w a visited & unvisited vertex.
(solid)

Back edges $\Rightarrow$ edges that connect vertices to previously visited vertices other than the immediate predecessor in the traversal
(dotted)

Algorithm DFS(G)

// implements DFS traversal on G

// input: graph $G, G = \{V, E\}$

// output: graph with vertices marked with consecutive integers in order they have been encountered.

Mark each vertex in V with 0 as a mark of being unvisited.

count $\leftarrow 0$

for each vertex in V

if v is marked with 0
dfs(v)

Algorithm dfs(v) $\rightarrow$ recursive

// visits all the unvisited vertices connected to vertex v & assigns a no. to them in order they are encountered

count $\leftarrow$ count + 1

mark v with count

for each vertex w in V adjacent to v do

if w is marked with 0 do
dfs(w)

Pseudocode:

DFS(G)

{

for ($i=1; i \leq n; i++$)

visit [i] = 0;

for ($i=1; i \leq n; i++$)

if (visit [i] == 0)

dfs(i)

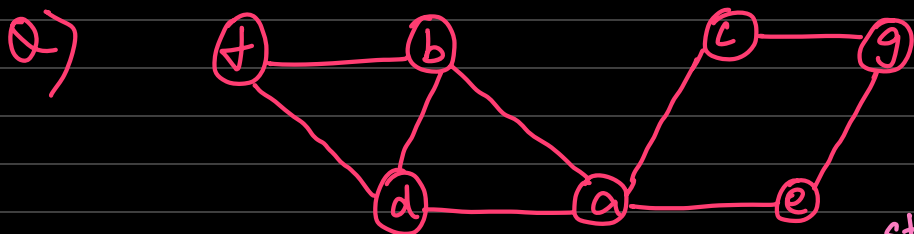
}

dfs(i)

}

BFS

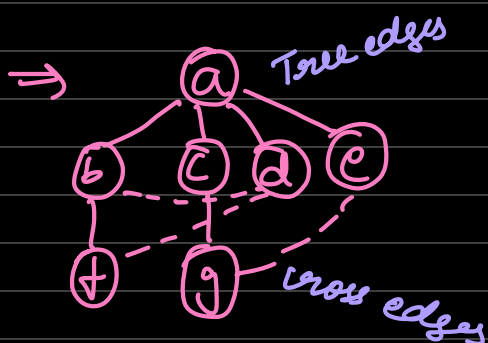
- 1) init a queue, insert starting vertex onto the queue and mark as visited
- 2) until queue $\Rightarrow$ empty, delete a node & find the neighbours of deleted node which are not visited, insert all of them to queue & mark as visited.



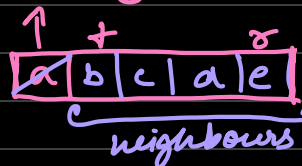
f $\rightarrow$ front end
e $\rightarrow$ rear end

$\Rightarrow$ (whatever is deleted comes here)

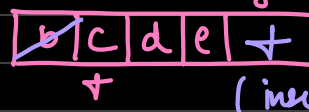
a₁
b₂
c₃
d₄
e₅
f₆
g₇



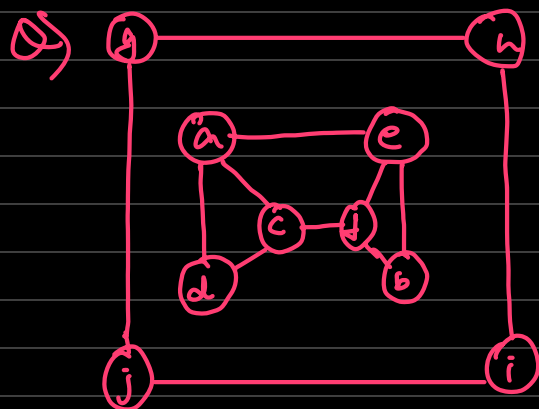
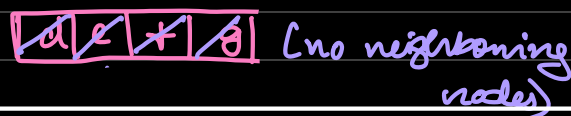
starting vertex



(insert neighbours of a)



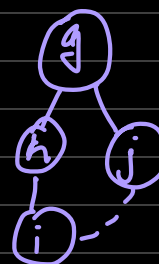
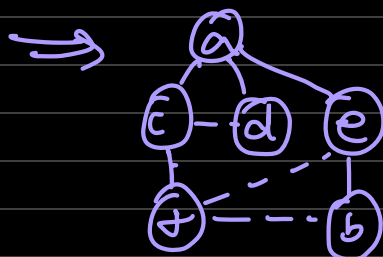
(insert neighbours of b)



$\Rightarrow$

a c d e
 f d e f
 d e f b
 f b
 g h j
 h j i
 j i

a₁
c₂
d₃
e₄
f₅
b₆
g₇
h₈
j₉
i₁₀



Algorithm BFS (G) Same as dfs

// implements BFS traversal on G

// input: graph G , $G = \{V, E\}$

// output: graph with vertices marked with consecutive integers in order they have been encountered.

Mark each vertex in V with 0 as a mark of being unvisited.

count $\leftarrow 0$

for each vertex in V

if v is marked with 0

bfs (v)

Algorithm bfs (v) $\rightarrow$ recursive

// visits all the unvisited vertices connected to vertex v & assigns a no. to them in order they are encountered

count $\leftarrow$ count + 1

mark v with count and initialize a queue

for each vertex w in V adjacent to v do

if w is marked with 0 do

count $\leftarrow$ count + 1

mark w with count

add w to queue

remove the v from queue

$v \rightarrow$ front vertex

Time complexity of DFS & BFS:

$\Rightarrow O(|V|^2)$ if input is adjacency matrix

$\Rightarrow O(|V| + |E|)$ if input is adjacency list.

Topological Sorting: (not a sorting alg)

⇒ application of DFS traversal.

⇒ Topological sorting is the ordering of directed acyclic graph (DAG) i.e. a graph that is not cyclic in a linear order for all nodes in graph.

⇒ If there is an edge from node x to y , x should be placed before y .

⇒ 2 methods to obtain topological ordering:

1. DFS method.
2. Vertex deletion method / Source removal method.

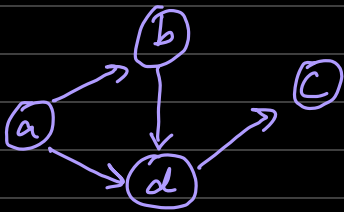
I DFS Method:

Step ①: traverse graph using DFS method.

Step ②: note the order in which the nodes became the dead end i.e. note the popping order of nodes.

Step ③: reverse the popping order, we will get topological order.

ex:

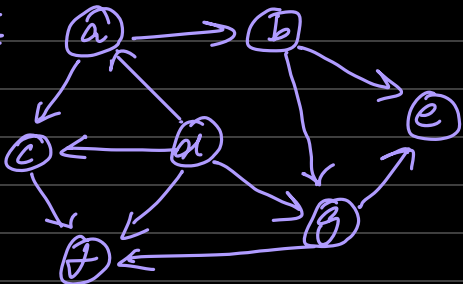


⇒
a 1, 4
b 2, 3
d 3, 2
c 4, 1

⇒ popping order: c d b a

⇒ Topological order: a b d c //

ex:



⇒
a 1, 6
b 2, 4
c 3, 1
g 4, 3
f 5, 2
e 6, 5
d 7, 7

⇒ pop order: e f g b c a d

⇒ top order: d a c b g f e //

II vertex deletion method:

indegree $\Rightarrow$ no. of incoming nodes

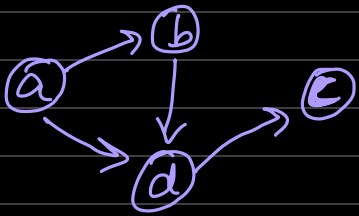
Step ①: note down indegree of all nodes.

Step ②: place the node/nodes that has indegree of zero.

Step ③: decrement the indegree of those nodes where there was an incoming node placed in step ② (deleting the vertex).

Step ④: repeat step ② and step ③ until all nodes are placed.

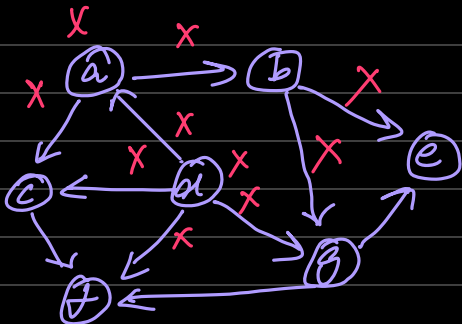
ex:



$\Rightarrow$ degree of a $\Rightarrow$ $in(a) = 0$ for a
 $in(b) = 1 \Rightarrow 1-1=0$ ✓ for b
 $in(d) = 2 \Rightarrow 2-1=1 \Rightarrow 1-1=0$ ✓ for d
 $in(c) = 1 \Rightarrow 1-1=0$ ✓ for c

$\Rightarrow$ Top order $\Rightarrow$ a (indegree of 0) (decrement the deg of nodes that have edge from a)
 b (indegree of 0) (" " b)
 d (indeg = 0) (" " d)
 c (no incoming nodes)

ex:



$\Rightarrow in(a) = 1-1=0 \Rightarrow$ Top order:
 $in(b) = 1-1=0$
 $in(c) = 2-1=1$ d a b c g e f
 $in(d) = 0$
 $in(e) = 2-1=1$
 $in(f) = 3-1=2-1=1$
 $in(g) = 2-1=1-1=0$

(keep erasing the nodes in diag while del it $\Rightarrow$ easier)

